GENERATED DOCUMENT

Technical Requirements Document

Newsletter Builder

Version 0.1.0 · commit 9abc092

November 6, 2026

Generated from source code

Reverse-engineered, read-only, from source code, config files, the automated test suite, `docs/PROGRESS.md`, and the existing `docs/brd/` / `docs/prd/`. No source file was changed to produce this document. Tagging convention: **Fact** (default — stated directly in code/config/tests), **Inference:** (a conclusion drawn from behavior, not stated verbatim), **Not Found in Codebase** (looked for, confirmed absent). See `VERIFICATION_NOTES.md` for every Inference/Not-Found item.

Verified at write time: `npx tsc --noEmit` → 0 errors. `npm test` → 41 test files, 950 tests, all passing.

Table of Contents

Part 1 — TRD-01-architecture.md

1. System Architecture Overview
2. Technology Stack & Dependencies
3. Project & Module Structure
4. Data Model & Type System

Part 2 — TRD-02-runtime-and-persistence.md

5. State Management (Zustand Store)
6. Rendering & Export Pipeline
7. Persistence Layer
8. Business Rule / Quality Engine

Part 3 — TRD-03-ai-and-components.md

9. AI Integration Architecture
10. Component Architecture
11. Build, Test & Tooling

Part 4 — TRD-04-security-risks-and-appendix.md

12. Security Architecture
13. Performance Characteristics
14. Technical Risks, Debt & Constraints
15. Developer Environment & Workflow
16. Glossary

Verification Notes

Every Inference and Not-Found-in-Codebase item, by section, with reasoning.

One-paragraph orientation

Newsletter Builder is a client-only React/TypeScript SPA (Vite, Zustand+Immer, no backend) with a bring-your-own-key AI subsystem calling OpenAI directly from the browser. This TRD covers its architecture, exact data model (quoted from `src/types.ts`), state-management and rendering pipeline, the complete `localStorage`/IndexedDB key inventory, the AI integration's security boundaries (CORS constraints, key redaction, structural secret-exclusion from backups), component patterns, and the build/test tooling — including this repository's own `scripts/docs-to-pdf/` documentation-build tool (§15.2), which is what turns this TRD into `docs/trd/pdf/TRD.pdf`.

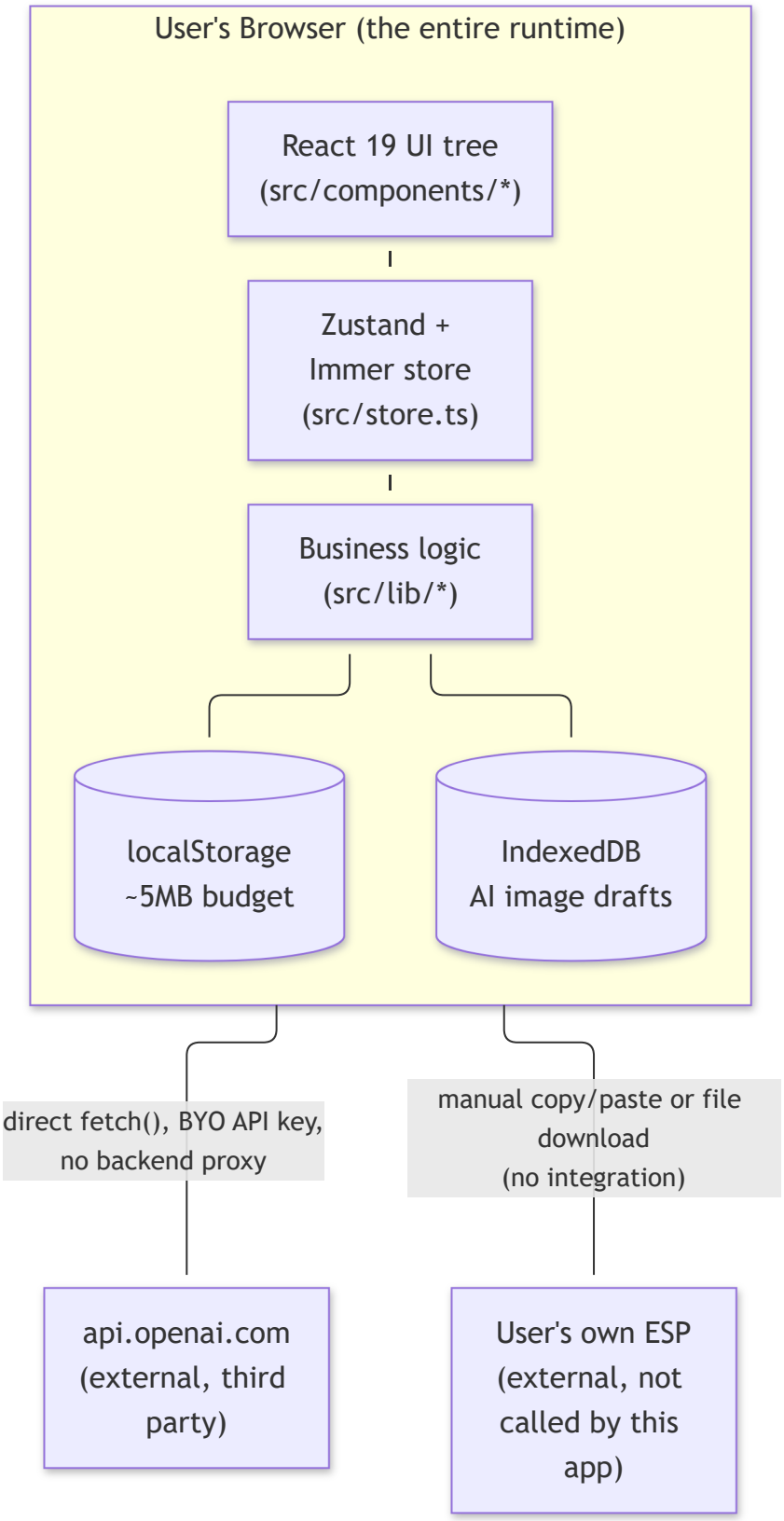
Part 1 of 4 — Sections 1–4: Architecture, Stack, Module Map, Data Model

---

1. System Architecture Overview

1.1 Shape of the system

Newsletter Builder is a **client-only single-page application**. There is no application server, no API gateway, no database server, and no deployed backend process anywhere in this repository. (Fact — confirmed by `vite.config.ts` containing no `server.proxy`/`server` key, and by the complete absence of any server framework — Express, Fastify, Next.js API routes, etc. — in `package.json`.)


User's Browser (the entire runtime)
React 19 UI tree
(src/components/*)
Zustand +
Immer store
(src/store.ts)
Business logic
(src/lib/*)
localStorage
~5MB budget
IndexedDB
AI image drafts
direct fetch(), BYO API key,
no backend proxy
manual copy/paste or file
download
(no integration)
api.openai.com
(external, third
party)
User's own ESP
(external, not
called by this
app)

1.2 Why this shape (Inference, consistent with the explicit design notes in `docs/PROGRESS.md`)

Feature 5A's own development notes record that **browser-callability of the OpenAI API was verified via a live, unauthenticated CORS preflight check before any AI code was written** — `OPTIONS https://api.openai.com/v1/images/generations` returned `access-control-allow-origin: *` with an allow-headers list of exactly `authorization,content-type`. This single fact is architecturally load-bearing: it is *why* the AI features could be built with no backend proxy at all, and it directly constrains the request shape (no `OpenAI-Organization` header, no telemetry header — either would fail the real preflight).

1.3 Deployment model

Not Found in Codebase: any CI/CD pipeline configuration, Dockerfile, infrastructure-as-code, hosting configuration, or deployment script. `npm run build` (`tsc --noEmit && vite build`) produces a static `dist/` bundle (confirmed present in the working tree at the time of writing) suitable for any static host, but **no evidence exists in this repository for which host, if any, is actually used in practice.**

1.4 Runtime dependency graph (external services)

External dependency	Required for	Failure mode if unreachable
`api.openai.com` (OpenAI REST API)	AI image generation (F-017 in the PRD), AI brief generation (F-018)	Every other feature continues to work; AI panels show a classified connection error
None else	—	—

---

2. Technology Stack & Dependencies

2.1 Core stack (Fact — `package.json`)

Layer	Technology	Version (as installed)
Language	TypeScript	`^7.0.2`
UI framework	React	`^19.3.1`
Build tool / dev server	Vite	`^8.3.5`
Styling	Tailwind CSS (via `@tailwindcss/vite`)	`^4.3.3`
State management	Zustand + Immer	`^5.0.16` / `^11.2.9`
Rich text editing	TipTap (`@tiptap/react`, `starter-kit`, `extension-link`, `extension-color`, `extension-text-align`, `extension-text-style`)	`^3.32.0`
Drag and drop	`@dnd-kit/core`, `@dnd-kit/sortable`, `@dnd-kit/utilities`	`^6.3.1` / `^10.0.0` / `^3.2.2`
HTML sanitization	DOMPurify	`^3.4.17`
Icons	`lucide-react`	`^1.62.0`
Test runner	Vitest (+ `jsdom` environment)	`^5.0.5` / `^30.1.3`
Component testing	`@testing-library/react`, `/dom`, `/user-event`	`^16.3.3` / `^10.4.2` / `^14.6.8`

2.2 Why each major dependency exists (Inference, from usage patterns observed across the BRD/PRD research)

- **Zustand + Immer:** one global store (`src/store.ts`) holding the design, brands, undo/redo stacks, and all transient UI state; Immer lets mutations be written as plain object edits while the store still returns new references for React's change detection.
- **dnd-kit:** powers canvas drag-and-drop (reordering blocks/rows, dragging a left-panel tile onto the canvas, dragging a saved block/row).
- **TipTap:** the Text block's rich-text editor; its extension set is narrow and deliberate (link, color, text-align, text-style) rather than the full starter kit's complete feature surface, matching the editor's actual exposed toolbar.
- **DOMPurify:** sanitizes block HTML before it is painted into the (non-iframe) canvas, which is also why the canvas cannot host a `<style>` block for dark-mode CSS (DOMPurify strips it) — a fact with a direct architectural consequence documented in `docs/PROGRESS.md` Feature 3B.
- **No routing library** (`react-router` or equivalent) is a dependency. **Fact**, confirmed by `package.json`; the application has exactly one route, by construction, not by omission.

2.3 Build/type configuration (Fact — `tsconfig.json`, `vite.config.ts`)

Setting	Value	Technical implication
`strict`	`true`	Full TypeScript strict-mode checking
`noUnusedLocals` / `noUnusedPa-rameters`	`true`	A build fails on dead variables/params
`noFallthroughCasesInSwitch`	`true`	Directly relevant to `render.ts`'s large block-type `switch` — an un-handled or fallen-through case fails the build
`isolatedModules` / `noEmit`	`true`	`tsc` is type-checking only; Vite/esbuild does the actual transpilation
`target` / `module` / `moduleReso-lution`	`ES2022` / `ESNext` / `bundler`	Modern, Vite-native module resolution
`jsx`	`react-jsx`	New JSX transform — no manual `React` import needed per file
Vite plugins	`@vitejs/plugin-react`, `@tailwindcss/vite`	React Fast Refresh + first-party Tailwind v4 integration
Vitest `environment`	`jsdom`	DOM APIs (including `DOMParser`, used by the renderer's sanitizer) are available under test
Vitest `include`	`src/test/**/*.test.{ts,tsx}`	The sole test-file glob — confirms the full test surface lives under `src/test/`

Not Found in Codebase: a separate `tsconfig.node.json`/`tsconfig.app.json` split (one unified `tsconfig.json` governs both app source and `vite.config.ts` itself, per its own `include` list); a dedicated lint script (`tsc --noEmit` is the only static-analysis gate — confirmed by `package.json`'s `scripts` block having no `lint` entry).

---

3. Project & Module Structure

3.1 Top-level layout (Fact)

```
src/
  main.tsx              entry point; first-run seeding (Welcome preset + Vee brand)
  App.tsx               root layout, keyboard shortcuts, auto-save/checks side effects
  store.ts              the one Zustand+Immer store — every mutation goes through it
  types.ts              the full Design/Block/Brand type system
  index.css             Tailwind entry + canvas-mirrored CSS (e.g. .eb-col, .eb-hide-mobile)
  components/           every screen, modal, panel, and property editor (flat, plus
settings/)
  lib/                  all business logic — no React imports anywhere in this directory
  lib/ai/               the AI subsystem — providers, key storage, task routing, brief
pipeline
  test/                 41 test files, one Vitest config (via vite.config.ts), jsdom
environment
```

3.2 `src/lib/*` — responsibility map (Fact, one line each, file names exact)

File	Responsibility
`defaults.ts`	Block-type defaults, `BLOCK_META`, `ROW_LAYOUTS`, font stacks
`design-ops.ts`	Tree lookups (`findBlock` / `findColumn` / `findRowIndex`), `applyRowLayout`, `normalizeDesign` (the one JSON-load gate)
`render.ts`	The single HTML generator — canvas and export share it
`output.ts`	The export pipeline boundary (`resolveForOutput`): brand resolve → AI-draft placeholder swap → UTM → merge-tag format
`brand.ts`	Brand CRUD helpers, token resolution, `resolveBrand`, `canEditBrand` / `getAccessibleBrands` (permission seams)
`dark.ts`	Dark-palette derivation, token-vs-literal diffing, class minting
`presets.ts`	The 8 starter preset definitions (`Preset.build(brand): Design`)
`merge.ts`	Canonical `{{tag}}` storage, the 4 ESP output-format tables
`utm.ts`	UTM tagging rules and skip conditions
`plaintext.ts`	HTML-design-to-plain-text conversion
`blocks-library.ts`	Saved-block/row validation, cloning, import planning

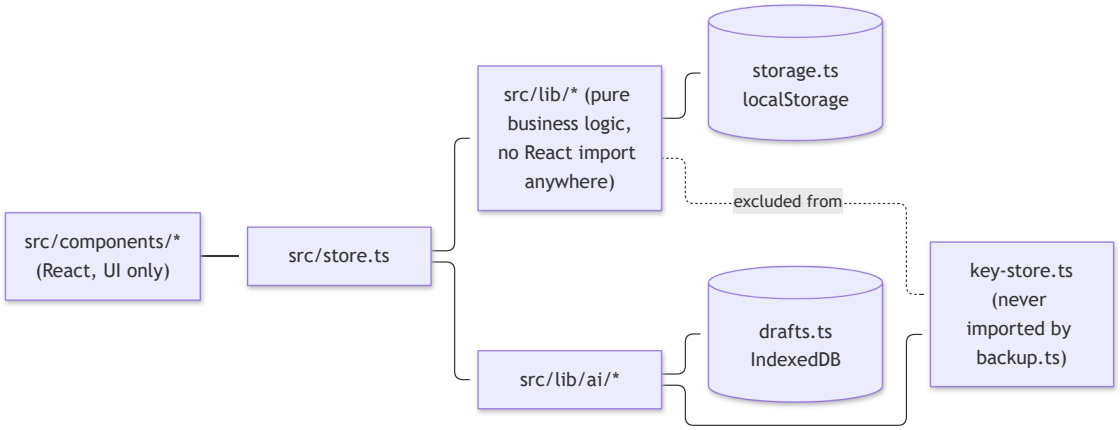
File	Responsibility
`versions.ts`	Pure version-history rules (snapshot/dedup/prune), no storage I/O
`storage.ts`	All `localStorage` reads/writes, `writeSafe()` quota recovery, schema versioning
`image.ts`	User-uploaded logo/image downscaling (unrelated to AI image generation)
`image-storage.ts`	The URL-based image library — validation, probing, the storage adapter interface
`image-audit.ts`	Design-level image collection and resolution-threshold notes
`audit.ts`	Placeholder-image URL pattern detection
`checks.ts`	The quality-rule engine — `checkDesign()` → `Issue[]`
`contrast.ts`	Pure WCAG 2.1 contrast math
`drafts.ts`	AI-generated image byte storage (IndexedDB abstraction)
`backup.ts`	Full-data export/import, deliberately excluding AI keys/drafts
`clear-all.ts`	The one function allowed to delete AI keys and drafts, for wipe purposes
`email-defaults.ts`	Org-wide defaults applied only at new-email creation
`user.ts`	The local, unauthenticated "profile" display name
`util.ts`	`uid()`, `slugify()`, `downloadFile()`, `safeUrl()`

3.3 `src/lib/ai/*` — responsibility map (Fact)

File	Responsibility
`provider.ts`	Shared image-provider contract, `AiError` type, `redactKeys()`, size/time limits
`openai.ts`	OpenAI image-generation adapter — request shape, size selection, error classification
`openai-text.ts`	OpenAI text/structured-output adapter
`text-provider.ts`	Shared text-provider contract
`validate.ts`	Image-response byte validation (magic bytes, size cap, base64, PNG dimensions)
`key-store.ts`	API key storage (memory/localStorage), remember toggle, connection status
`styles.ts`	Image style-preset suffixes

File	Responsibility
`tasks.ts`	The 2-entry task registry (`text.brief`, `image.generate`)
`task-settings.ts`	Per-task routing persistence, fallback chains, one-time migration
`run-task.ts`	The single call-site function — routing, per-task guard, fallback-on-error
`model-access.ts`	Session-only, per-task "model access denied" signal
`text-consent.ts`	Per-provider consent-granted flag for sending brief text to a text provider
`slots.ts`	The slot-derivation pattern — classifies a preset's block tree into AI-fillable fields
`brief-types.ts`	`BriefFields` / `BriefRecord` shapes and defaults
`brief-prompt.ts`	System/user prompt construction, prompt-injection framing
`brief-schema.ts`	Strict-mode JSON Schema builder for the brief response
`brief-validate.ts`	Sanitization + shape/limit validation of the AI response
`create-from-brief.ts`	Orchestration: generate draft → write onto design
`preset-selection.ts`	Auto preset resolution by brief goal

3.4 Module dependency direction (Inference, drawn from the import graph implied by the responsibility maps above)



The one architecturally enforced exclusion worth diagramming explicitly: `backup.ts` **never imports** `key-store.ts` or `drafts.ts` — a structural (not just behavioral) guarantee that a full-data export cannot reach either, confirmed by reading that file's own import list rather than by testing its runtime output alone.

---

4. Data Model & Type System

All of the following interfaces are quoted directly from `src/types.ts` (field sets condensed to the technically load-bearing ones; optional/derived fields noted).

4.1 The design tree

```
interface Design {
  version: 1;
  brandId: string | null;
  body: BodySettings;
  rows: Row[];
}

interface Row {
  id: string;
  layout: number[];           // column ratios, e.g. [1, 2]
  bgColor: string;            // the ONLY background that can span the full window
  contentBgColor: string;
  padding: Padding;
  stackOnMobile: boolean;
  dark?: DarkOverrides;        // manual per-row dark override
  columns: Column[];
}

interface Column {
  id: string;
  blocks: Block[];
}
```

4.2 `Block` — a discriminated union, 13 types

```
type BlockType =
  | 'heading' | 'text' | 'image' | 'button' | 'divider'
  | 'menu' | 'social' | 'video' | 'html'
  | 'header' | 'hero' | 'footer' | 'articles';

interface Block {
  id: string;
  type: BlockType;
  values: /* one of 13 type-specific *Values shapes, each also carrying
             padding: Padding and an optional dark?: DarkOverrides */;
}
```

Several `*Values` shapes additionally carry `aiDraftId?: string` (a pointer to a locally-generated image held in this browser's IndexedDB — never image bytes) and/or `noTrack?: boolean` (a per-link UTM opt-out).

4.3 Email-wide settings

```
interface BodySettings {
  bgColor: string; contentBgColor: string; width: number; fontFamily: string;
  textColor: string; linkColor: string; preheader: string; title: string;
  showFooter: boolean;
  mergeFormat: 'curly' | 'mailchimp' | 'hubspot' | 'custom';
  mergeCustom: { prefix: string; suffix: string };
  utm: UtmSettings;
  darkMode: boolean;
}

interface UtmSettings {
  enabled: boolean; source: string; medium: string; campaign: string;
  content: string; term: string;
}
```

4.4 Brand kit

```
interface Brand {
  id: string; name: string; slug: string;
  logo: string; logo_dark: string;           // uploaded, canvas-preview only
  logo_url: string; logo_dark_url: string;   // hosted https, the ONLY thing exports use
  website_url: string;
  tokens: BrandTokens;
  darkTokens: BrandDarkColors | null;        // null = "derive one"
  voice: BrandVoice | null;                  // null = "no voice guidance set"
  footer: BrandFooter;
  social_links: BrandSocialLink[];
  is_active: boolean;
  createdAt: number; updatedAt: number;
  createdBy: string | null; updatedBy: string | null;  // local display-name label, not auth
  schemaVersion: number;
}

interface BrandTokens {
  colors: { primary: string; secondary: string; accent: string; text: string;
            muted: string; link: string; page_bg: string; content_bg: string; border: string
};
  fonts: { heading: string; body: string };
  button: { radius: number; paddingV: number; paddingH: number;
            weight: 'normal' | 'bold'; uppercase: boolean };
}

interface BrandDarkColors {
  page_bg: string; content_bg: string; text: string; muted: string;
  link: string; border: string; button_bg: string; button_text: string;
}

interface BrandVoice {
  description: string; wordsToUse: string[]; wordsToAvoid: string[];
}
```

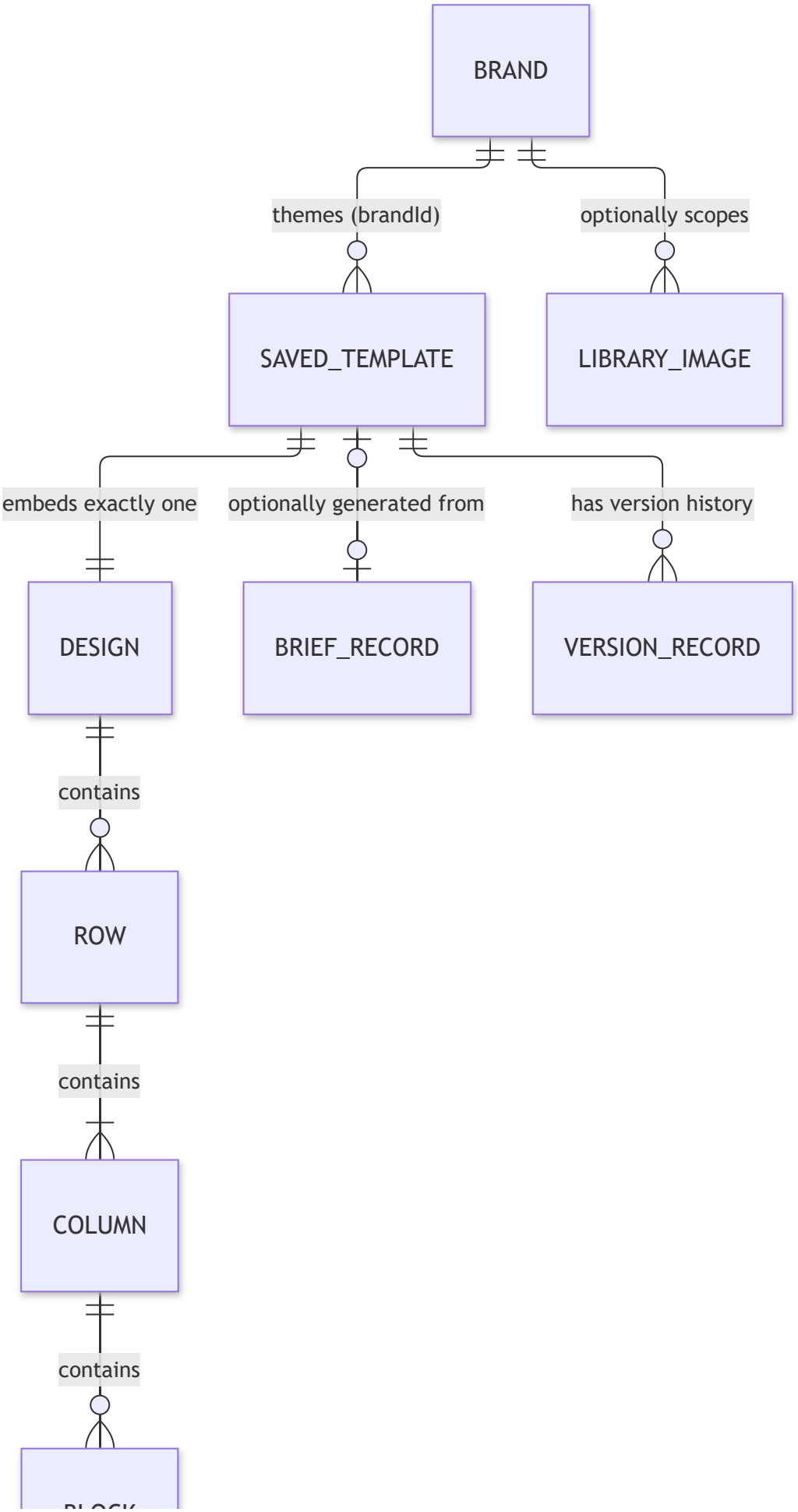
4.5 Editor-only / selection state

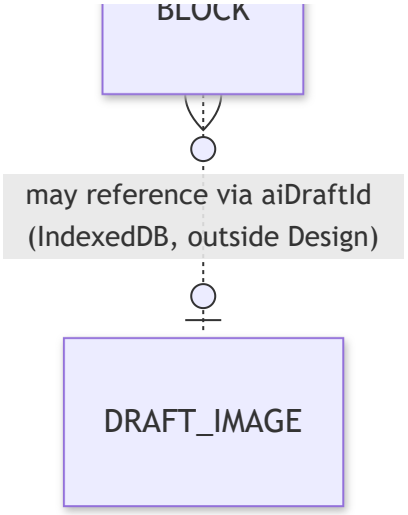
```
type Selection = { kind: 'body' } | { kind: 'row'; id: string } | { kind: 'block'; id: string
};
type Device = 'desktop' | 'mobile';
type Scheme = 'light' | 'dark';
```

4.6 Records defined outside `types.ts` (Fact — storage/library modules)

Type	Defined in	Notes
`SavedTemplate`	`src/lib/storage.ts`	`{ id, name, updatedAt, createdAt, brandId, design: Design, autoSaved: boolean, createdBy, updatedBy, schemaVersion, brief: BriefRecord | null }`
`SavedBlock`	`src/lib/blocks-library.ts`	`{ id, name, category, kind: 'block' | 'row', payload: Block | Row, createdAt, updatedAt, schemaVersion }`
`LibraryImage`	`src/lib/image-storage.ts`	`{ id, url, name, defaultAlt, tags[], width, height, brandId: string | null, createdAt, status: 'ok' | 'broken' | 'unknown', checkedAt, schemaVersion }`
`VersionRecord`	`src/lib/versions.ts`	`{ id, emailId, at, note, reason, rows, blocks, design: Design, schemaVersion }` — never includes a `Brand`
`BriefRecord`	`src/lib/ai/brief-types.ts`	`{ fields: BriefFields, resolvedPresetId, reviewed: boolean, generatedAt }`

4.7 Entity relationships


BRAND
themes (brandId)
optionally scopes
SAVED_TEMPLATE
LIBRARY_IMAGE
embeds exactly one
optionally generated from
has version history
DESIGN
BRIEF_RECORD
VERSION_RECORD
contains
ROW
contains
COLUMN
contains



Part 2 of 4 — Sections 5–8: State Management, Rendering Pipeline, Persistence, Quality Engine

---

> Continues from `TRD-01-architecture.md`.

---

5. State Management (Zustand Store)

5.1 Store shape (Fact — `src/store.ts`)

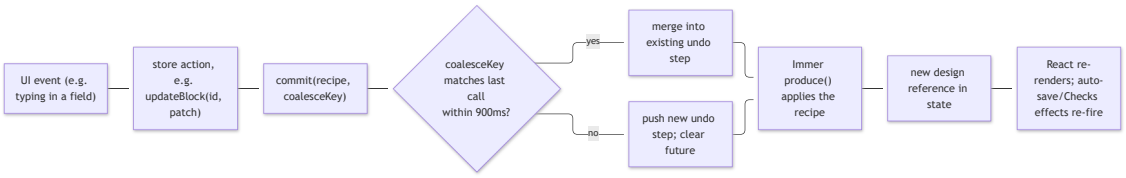
One store, `useEditor`, created via `zustand` + `immer`'s `produce`. Selected state fields:

Field	Type	Purpose
`design`	`Design`	The currently edited email
`brands`	`Brand[]`	Every brand on this browser, archived included
`past` / `future`	`Design[]`	Undo/redo stacks, capped at `HISTORY_LIMIT = 100`
`selection`	`Selection`	Current editor selection
`device` / `scheme`	`Device` / `Scheme`	Canvas preview toggles
`revision`	`number`	Bumped on any whole-design **load** (open/restore/undo) — the signal auto-save uses to distinguish a load from an edit
`currentEmailId`	`string | null`	The saved-email record this canvas belongs to; `null` until first save
`dirty`	`boolean`	A change the debounce has not yet written
`saveState`	`{status, at, message}`	Top-bar save status, never silently idle after a failure
`savedBlocks` / `images`	`SavedBlock[]` / `LibraryImage[]`	The two reusable libraries
`issues` / `ignoredIssues`	`Issue[]` / `string[]`	Checks engine output and per-email dismissals
`knownDrafts` / `draftsChecked`	`string[]` / `boolean`	Which AI-draft ids this browser actually holds
`currentBrief`	`BriefRecord | null`	The brief this email was generated from, if any

5.2 The mutation gateway

Every structural/value mutation flows through an internal `commit(recipe, coalesceKey?)` helper, which:

1. Applies the recipe via Immer's `produce()` — mutations are written as plain edits, the result is a new, referentially-distinct object.
2. Pushes the **previous** design onto `past` (capped, oldest dropped past 100) and clears `future`.
3. If `coalesceKey` matches the previous call's key within `COALESCE_MS = 900` ms, merges into the existing undo step instead of creating a new one.



5.3 Brands are explicitly outside undo history

Brand writes (`upsertBrand`, `duplicateBrand`, `setBrandArchived`) go through `writeBrands()` → `set({brands: next})` directly, never through `commit()` — a documented, deliberate exception, since brands are shared state across every email in the browser, not scoped to the currently edited document.

---

6. Rendering & Export Pipeline

6.1 One renderer, two consumers

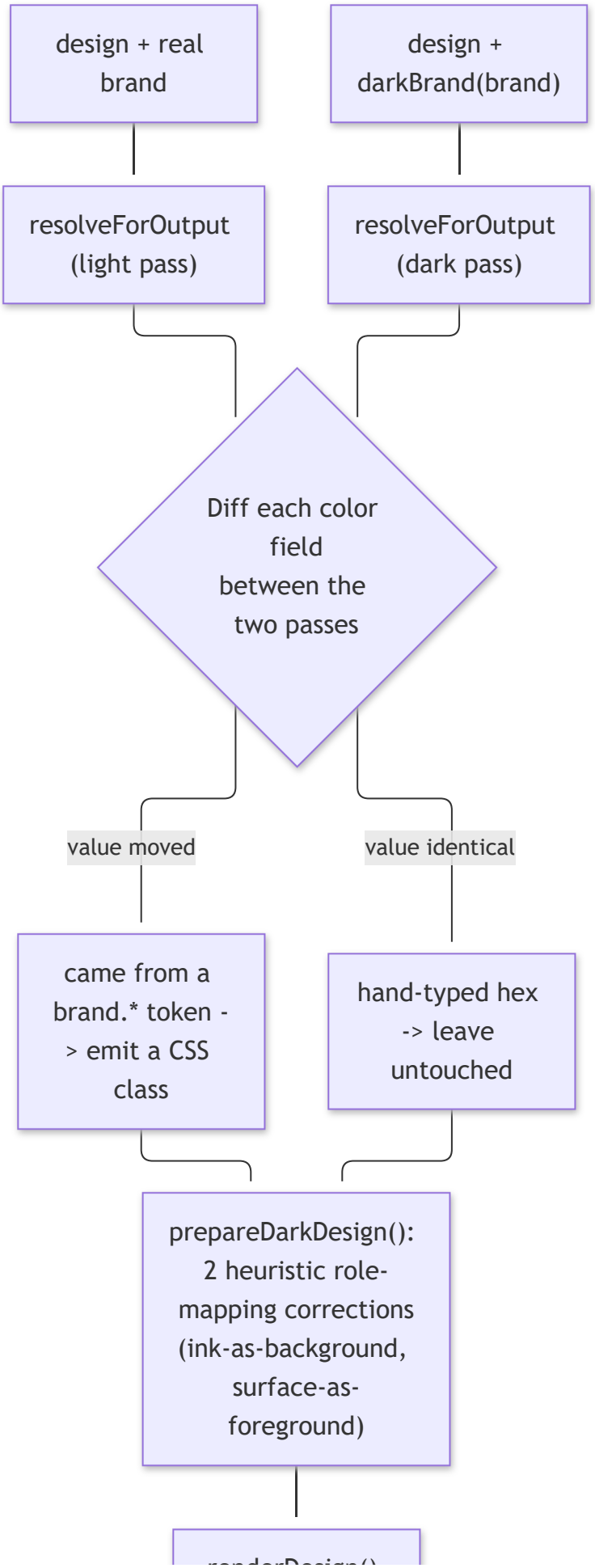
`renderBlock()` / `renderBlockInner()` in `src/lib/render.ts` is the **single** source of HTML for both the live canvas and every export path. This is enforced by construction — there is exactly one function, not two implementations kept in sync by convention.

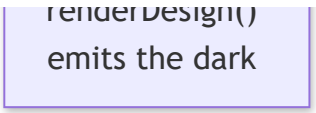
6.2 The export boundary

```
// src/lib/output.ts — resolveForOutput(), simplified to its actual call order
function resolveForOutput(design: Design, brand: Brand, mode: 'export' | 'preview', opts?) {
  let out = resolveDesign(design, brand, mode);   // brand tokens -> literals
  out = substituteDraft(out, opts?.draftImages);   // aiDraftId -> placeholder URL (or real
bytes, canvas/preview only)
  if (mode === 'export') out = addUtm(out, design.body.utm); // export-only
  out = formatMergeTags(out, design.body.mergeFormat);
  return out; // caller then calls renderDesign() and/or toPlainText()
}
```

Order is load-bearing: UTM tagging runs while merge tags are still canonical `{{tag}}`, so "is this URL nothing but a merge tag" stays one regex rather than a per-ESP-format problem.

6.3 Dark-mode double-resolve


design + real brand
design + darkBrand(brand)
resolveForOutput (light pass)
resolveForOutput (dark pass)
Diff each color field between the two passes
value moved
value identical
came from a brand.* token -> emit a CSS class
hand-typed hex -> leave untouched
prepareDarkDesign(): 2 heuristic role-mapping corrections (ink-as-background, surface-as-foreground)



With `body.darkMode` false, `darkRenderOpts()` returns `{}` and no dark machinery runs at all — this single branch is why the byte-identical-export guarantee (checked by snapshot tests) holds.

6.4 Normalization — the one JSON-load gate

`normalizeDesign(input: unknown): Design | null` (`src/lib/design-ops.ts`) is reused by every entry point that reads untrusted/stored JSON: `storage.ts` (draft/template load), `versions.ts` (version load), `backup.ts` (restore). It returns `null` only if `input.rows` is not an array; otherwise unknown block types are silently dropped, missing fields are filled from `createBlock(type)` defaults, and known pre-feature migrations are applied explicitly (e.g. a design with no `showFooter` gets `true`; no `darkMode` gets `true`).

---

7. Persistence Layer

7.1 `localStorage` key inventory (Fact — confirmed against `src/lib/storage.ts` and the AI subsystem's own storage modules)

Key	Schema version field	Contents
`email-builder:draft:v1`	n/a (legacy)	Pre-2B single "open email" slot; migrated once then deleted
`email-builder:templates:v1`	`SavedTemplate.schemaVersion`	Every saved/auto-saved email
`email-builder:brands:v1`	per-`Brand.schemaVersion` (currently 3)	All brand kits
`email-builder:sample:v1`	none	Merge-tag preview sample values — editor-only, never in the `Design`
`email-builder:blocks:v1`	`SavedBlock.schemaVersion` (currently 2)	Saved-block/row library
`email-builder:versions:v1`	`VersionRecord.schemaVersion`	All version snapshots, every email, one key (so quota-pruning can free space globally)
`email-builder:session:v1`	stamped	`{ lastEmailId }` — which email to reopen on next load
`email-builder:images:v1`	`LibraryImage.schemaVersion` (currently 2)	Image library
`email-builder:ignored:v1`	stamped	`{ byEmail: Record<emailId, string[]> }` — dismissed Checks issue ids
`email-builder:profile:v1`	stamped (currently 1)	`{ name, hintShown }` — local display-name identity
`email-builder:email-defaults:v1`	stamped (currently 1)	Org-wide new-email defaults
`email-builder:aikeys:v2` (+ legacy `:v1`, migrated once)	`KEY_SCHEMA` (currently 2)	`{ activeProviderId, byProvider: { [id]: { model, key } } }` — opt-in only
`email-builder:ai-task-routing:v1`	stamped (currently 1)	Per-task provider/model/options/fallback-chain config — **never a key**
`email-builder:ai-text-consent:v1`	none	Provider ids the user has consented to send brief text to

IndexedDB: database `email-builder-drafts`, object store `images`, keyed by id — holds `DraftImage { id, blob, mime, width, height, prompt, alt, model, createdAt }` (AI-generated image bytes; `memoryDraftStore()` substitutes a `Map` under test, selected via `typeof indexedDB === 'undefined'`).

7.2 Quota recovery

```
// src/lib/storage.ts — writeSafe(), the only writer anything outside this module should use
type WriteResult = 'ok' | 'pruned' | 'full';
function writeSafe(key: string, value: unknown): WriteResult {
  if (write(key, value)) return 'ok';
  const pruned = pruneOldestGlobal(listVersions(), QUOTA_PRUNE_COUNT /* = 10 */);
  if (pruned.length && write(VERSIONS_KEY, pruned) && write(key, value)) return 'pruned';
  return 'full';
}
```

`STORAGE_LIMIT_BYTES = 5 * 1024 * 1024` is a constant used only for the UI's usage readout — the real browser-enforced cap is not read programmatically (no API exposes it) and is only approximated. `storageUsage()` sums `(key.length + value.length) * 2` bytes (UTF-16 code units — what the quota is actually measured in) over every `email-builder:`-prefixed key.

7.3 Backup/restore, and the structural secret-exclusion guarantee

`backup.ts`'s `BackupData` covers exactly `{ brands, templates, savedBlocks, images, versions }`. The guarantee that the AI key store and drafts can never leak through a backup is **structural**: the file never imports `key-store.ts` or `drafts.ts` at all — a property verifiable by reading its import list, not merely by testing its output. `importBackup()` normalizes every record through the same `normalize*()` function its category's normal load path uses; a record that fails to normalize is dropped, not a reason to reject the whole import.

7.4 Schema versioning discipline

Each persisted category keeps its **own** independent version counter (`SCHEMA_VERSION` for templates = 2, `BRAND_SCHEMA_VERSION` = 3, `SAVED_BLOCK_SCHEMA` = 2, `IMAGE_SCHEMA` = 2) — not one global app-wide version. A record with no `schemaVersion` field is treated as version 1 and kept (never discarded for being old); every `normalize*()` function fills missing fields from current defaults rather than rejecting the record.

---

8. Business Rule / Quality Engine

8.1 Entry point

```
// src/lib/checks.ts
function checkDesign(
  resolvedDesign: Design,   // already brand-resolved, UTM-tagged, merge-formatted
  exportedHtml: string,     // the real rendered output
  brand: Brand,
  opts: { knownDrafts?: string[]; probedImages?: Record<string, 'ok'|'broken'> },
): Issue[]
```

Passing the **unresolved** design is documented in-code as a caller bug, not a condition the engine itself guards against at runtime.

8.2 Rule catalogue (Fact — exact codes, thresholds, and severities)

Group	Code	Severity	Condition
SIZE	`SIZE_NEAR_LIMIT`	warning	≥ 90 KB (UTF-8 bytes of the real HTML)
SIZE	`SIZE_OVER_LIMIT`	error	≥ 102 KB
SIZE	`SIZE_DARK_OVERHEAD`	warning	dark-mode CSS/markup adds > 3 KB, measured by double-rendering
SUBJECT	`SUBJECT_EMPTY`	error	`body.title` blank
SUBJECT	`SUBJECT_LONG`	info	> 60 chars
PREHEADER	`PREHEADER_EMPTY`	warning	`body.preheader` blank
PREHEADER	`PREHEADER_LONG`	info	> 110 chars
UNSUBSCRIBE	`UNSUBSCRIBE_MISSING`	warning	no "unsubscribe" string (any ESP format) found in the HTML
CONTENT	`CONTENT_THIN`	warning	< 20 visible words **and** ≥ 1 image
CONTRAST	`CONTRAST_TEXT` / `CONTRAST_BUTTON` / `CONTRAST_LINK`	error	WCAG AA not met against the effective background
CONTRAST	`CONTRAST_UNKNOWN`	info	text sits on a hero background image — unmeasurable, reported as such rather than guessed

Group	Code	Severity	Condition
CONTRAST (dark)	`CONTRAST_TEXT_DARK` / `CONTRAST_BUTTON_DARK` / `CONTRAST_LINK_DARK`	error	same AA check, run a second time on the dark-resolved design
READABILITY	`TEXT_TOO_SMALL`	warning	font-size < 12px
READABILITY	`TAP_TARGET_SMALL`	warning	effective button height < 44px
LINKS	`LINK_EMPTY` / `LINK_HASH` / `LINK_RELATIVE`	error	no href / `#` / relative URL
LINKS	`LINK_HTTP`	warning	`http://` not `https://`
LINKS	`LINK_PLACEHOLDER_DOMAIN`	warning	host matches example.com/example.org/example.net/yourdomain.com/yoursite.com/yourcompany.com or a subdomain
IMAGES	`IMAGE_EMBEDDED` / `IMAGE_NOT_HTTPS`	error	`data:` URI / not https
IMAGES	`IMAGE_BROKEN`	error	**only** once the URL has been actively probed and failed
IMAGES	`IMAGE_PLACEHOLDER`	warning	matches the known placeholder-domain pattern list
IMAGES	`IMAGE_MISSING_ALT`	warning	the field has an alt attribute slot and it is blank
IMAGES	`IMAGE_TOO_WIDE`	warning	natural width > 1600px (requires the image to be in the library, where natural size is measured)
DARK	`DARK_NO_LOGO`	info	the email uses a logo and the brand has no `logo_dark_url`
DARK	`DARK_IMAGE_MANUAL`	info	a hero centered background image, which dark mode cannot restyle
AI image	`AI_IMAGE_MISSING_DRAFT` / `AI_IMAGE_NOT_HOSTED`	error	an `aiDraftId` not present locally / present but nowhere public
AI brief	`AI_DRAFT_UNREVIEWED`	warning	a brief-generated email not yet marked reviewed

Group	Code	Severity	Condition
AI brief	`BRIEF_PLACEHOLDER_LEFT`	warning	a `[bracketed place-holder]` still present in copy

8.3 Stable issue identity

```
const id = `${code}#${blockId ?? 'email'}#${discriminator}`;
```

The same underlying problem keeps the same `id` across unrelated edits and across runs, which is the technical precondition for "Ignore" to mean anything persistent rather than coincidentally matching a re-computed list.

8.4 Contrast math (pure, independently testable)

`src/lib/contrast.ts`: sRGB → linear RGB → relative luminance → contrast ratio (range 1–21). "Large text" = font-size ≥ 24px, or ≥ 18.66px **and** bold. Required ratio: 4.5:1 normal, 3:1 large. `suggestFix()` walks the foreground color toward black or white (whichever direction increases contrast) in fixed steps until the threshold clears, which is the mechanical source of a concrete "Try #919191"-style suggestion in the UI.

Part 3 of 4 — Sections 9–11: AI Integration, Component Architecture, Build & Test Tooling

> Continues from `TRD-02-runtime-and-persistence.md`.

---

9. AI Integration Architecture

9.1 Request shape — image generation

```
// src/lib/ai/openai.ts — the ONLY two headers ever sent, confirmed against a
// live CORS preflight before this adapter was built
headers: {
  Authorization: `Bearer ${key}`,
  'Content-Type': 'application/json',
}
body: { model, prompt, size, n: 1, output_format: 'png', quality? } // quality omitted
entirely when unset
```

`response_format` is never sent (deprecated/unsupported for GPT image models — the response is always base64 `data[].b64_json`, so there is never a second cross-origin fetch of a returned URL). `REQUEST_TIMEOUT_MS = 90_000`, honoring both an internal timer and the caller's own `AbortSignal`.

9.2 Size selection logic

```
function sizeFor(aspect: AspectChoice, blockWidth: number, model: string): string {
  const arbitrary = /^gpt-image-2(\.\d+)?(-|$)/.test(model); // "arbitrary size" model family
  switch (aspect) {
    case '1:1': return '1024x1024';
    case '3:2': return '1536x1024';
    case '16:9': return arbitrary ? '1536x864' : '1536x1024';
    case '4:3':  return arbitrary ? '1360x1024' : '1536x1024'; // 1360 = nearest multiple of
16 to true 4:3
    case 'match': /* infer from blockWidth: >=480 landscape, <220 portrait (1024x1536), else
square */
  }
}
```

The `1360` constant is the fix for a real, previously-shipped bug: an earlier `1365x1024` value is not divisible by 16, which is a documented OpenAI constraint for this model family (both sides ÷16, ratio 1:3–3:1, 3840×2160 ceiling) — every 4:3 request silently failed and fell back to 3:2 until this was corrected.

9.3 Error classification (closed set)

```
type AiErrorKind =
  | 'invalid-key' | 'forbidden' | 'no-credit' | 'rate-limit'
  | 'content-policy' | 'network' | 'timeout' | 'bad-response' | 'cancelled';
```

Response	Classification
HTTP 401	`invalid-key`

Response	Classification
HTTP 403	`forbidden`
HTTP 429 + `insufficient_quota`	`no-credit`
HTTP 429, otherwise	`rate-limit`
HTTP 502/503/504/524	`network` (message explicitly warns the request "may well have been made and billed")
Body/code containing `content_policy`/"safety system"/"moderation"	`content-policy`
An HTML body where JSON was expected	`bad-response` ("a proxy or firewall returned a web page")
Empty body	`bad-response`
Anything else ≥500	`network`
`DOMException('AbortError')`	`cancelled`

A second, narrower check (`isModelAccessError()`) recognizes a model-access denial from either a `403` **or** message text matching `/do(?:es)? not have access|access to (this |the )?model|model_not_found/i` — added because some providers answer an inaccessible model with a generic 404 that `classify()` would otherwise bucket as plain `bad-response`.

9.4 Key handling — the security-relevant implementation detail

```
// src/lib/ai/key-store.ts
const memory = new Map<string, StoredEntry>();  // default: dies with the tab
// "Remember on this device" (opt-in, per provider) persists into:
//   localStorage: email-builder:aikeys:v2
// redactKeys(text, key) strips: the exact key (>=8 chars), any
//   sk-/rk-/org-/proj- prefixed token, and any "Bearer <token>" pattern,
// applied to every provider-echoed error string before it reaches the UI.
```

Deliberately not in the Zustand store. Every field in that store is a candidate for JSON serialization (designs, brands, saved blocks, and version snapshots all round-trip through it); keeping the key out of it structurally prevents any future feature that forgets this rule from accidentally writing it out. This guarantee is swept by dedicated tests across every serialization surface: the design, brands, saved blocks, image library, issues, the rendered export string, every `localStorage` entry, and `document.body.textContent`.

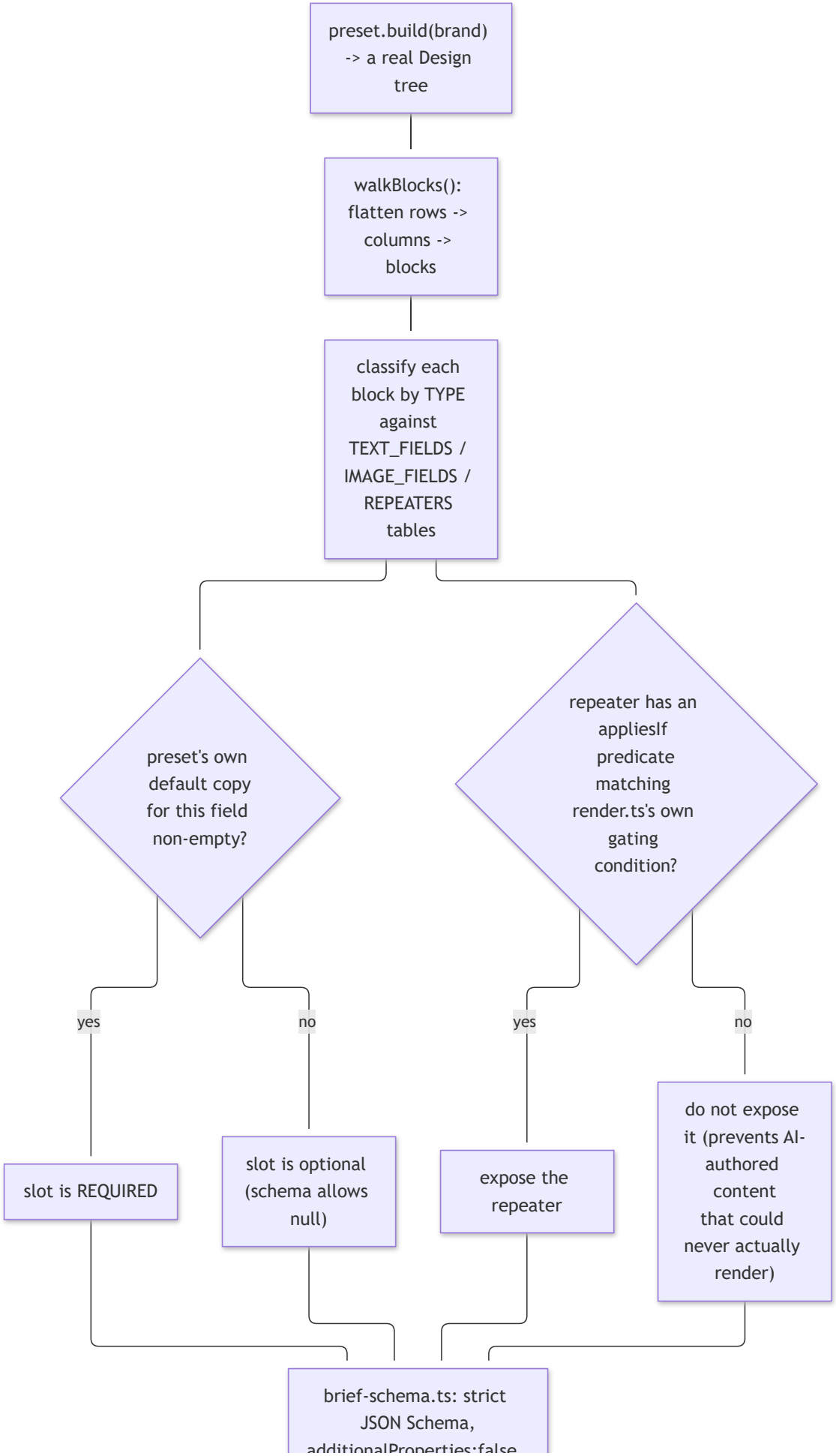
9.5 Per-task routing and fallback

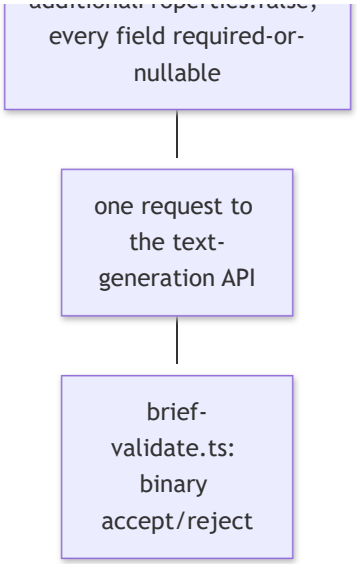
```
type AiTaskId = 'text.brief' | 'image.generate';
type TaskRoute = { providerId: string; model: string; options: object; fallbacks:
FallbackEntry[] };

// src/lib/ai/run-task.ts — the one call-site function
const FALLBACK_TRIGGER = new Set(['network', 'timeout', 'rate-limit']);
async function runTask<T>(taskId: AiTaskId, input): Promise<T> {
  for (const candidate of candidatesFor(taskId)) {          // primary, then each fallback in
order
    if (!candidate.isConnected) continue;                    // skip: no key, or not proven
this session
    try { return await candidate.call(input); }
    catch (err) {
      if (!FALLBACK_TRIGGER.has(classify(err))) throw err;    // bad key/content-policy/etc.
stop immediately
      // else: try the next candidate
    }
  }
  throw lastError;
}
```

A per-task, module-level `guards: Map<AiTaskId, GuardState>` enforces one in-flight call per task plus a shared `MIN_REQUEST_GAP_MS = 2_000` floor between consecutive calls for the same task.

9.6 Brief-generation pipeline — the "slot-derivation pattern"


preset.build(brand) -> a real Design tree
walkBlocks(): flatten rows -> columns -> blocks
classify each block by TYPE against TEXT_FIELDS / IMAGE_FIELDS / REPEATERS tables
preset's own default copy for this field non-empty?
repeater has an appliesIf predicate matching render.ts's own gating condition?
yes
no
yes
no
slot is REQUIRED
slot is optional (schema allows null)
expose the repeater
do not expose it (prevents AI-authored content that could never actually render)
brief-schema.ts: strict JSON Schema, additionalProperties:false



The `appliesIf` gate (step G/I) is a fix for a real, previously-shipped bug: `createBlock('hero')` always populates a default 2-item `cards` array regardless of layout, but `render.ts` only draws it for `layout: 'cards-2'/'cards-3'` — the first version of the slot-derivation code exposed that array as an AI-fillable field whenever non-empty (i.e. always), which would have asked the model to write hero-card copy that could never render for 7 of the 8 presets. The fix replicates the exact condition `render.ts` itself checks, rather than a weaker "is this field non-empty" test.

9.7 Prompt-injection defense

1. The brief and any pasted source text are each wrapped in `===== BEGIN/END <name> (untrusted data) =====` delimiters, with an explicit system-prompt instruction to treat their contents as data, never instructions.
2. A mechanical backstop: the strict JSON Schema (`additionalProperties: false`, every field required-or-nullable) constrains the model's output shape at the token level — injected text cannot add fields or escape the schema regardless of what it asks for.

---

10. Component Architecture

10.1 Top-level composition (Fact — `src/App.tsx`)

```
<ConfirmProvider>
  <TopBar />
  <DndShell>
    <LeftPanel />
    <Canvas />
  </DndShell>
  {aiTarget ? <AiImagePanel /> : <RightPanel />}
  <ChecksPanel />
  {/* modals rendered conditionally by a ModalName union, not a router */}
</ConfirmProvider>
```

There is no `react-router` and no history-API usage anywhere — confirmed by grep across `src/components/`. Every "screen" is a modal (`Modal` wrapper in `Modals.tsx`), an overlay (`ChecksPanel`), or a panel swap (`AiImagePanel` replacing `RightPanel`; `SettingsHub` replacing `LeftPanel`'s tab content).

10.2 Cross-component communication without prop-drilling or a router

A recurring pattern: a deeply nested component (e.g. an `ImageUrlField` three levels inside a block editor) needs to open a top-level panel without a prop path connecting them and without a new store slice. The solution used consistently across the codebase is a **module-level callback registry**:

```
// src/components/fields.tsx (5A) — reused verbatim by settings-nav.ts (6B)
let openImpl: ((target: X) => void) | null = null;
export function registerXOpener(fn: typeof openImpl) { openImpl = fn; }
export function openX(target: X) { openImpl?.(target); }
```

`App.tsx` registers the real implementation once on mount; any component anywhere can call the exported `openX()` function directly. `LeftPanel.tsx` re-registers its settings-opener on every active-tab change specifically so a later external "Open AI settings" call restores the *current* tab rather than a stale one captured at mount.

10.3 The "shared component, two mount points" pattern

`AiImagePanel.tsx` exports `SettingsView` as a named export; `AiSection.tsx` (inside the Settings hub) mounts that exact same component — not a reimplementation — so the AI image panel's own gear icon and Settings → AI are provably the same code and the same `key-store.ts` module state, never two copies that could drift apart.

10.4 Rendering parity between canvas and export

The canvas does **not** render inside an iframe — block HTML is sanitized via DOMPurify and painted into divs. This has a direct, documented architectural consequence: DOMPurify strips `<style>` tags, so

the canvas cannot host the dark-mode stylesheet, which is why the Light/Dark toggle only drives the **Preview** modal (which *is* a sandboxed iframe containing the literal exported HTML) and why the canvas itself is "always light," by construction, not by an unfinished feature.

---

11. Build, Test & Tooling

11.1 Scripts (Fact — `package.json`)

Script	Command	Purpose
`dev`	`vite`	Dev server
`build`	`tsc --noEmit && vite build`	Type-check, then production bundle to `dist/`
`preview`	`vite preview`	Serve the production build locally
`test`	`vitest run`	Full test suite, single run (no watch)
`typecheck`	`tsc --noEmit`	Type-check only
`docs:pdf`	`node scripts/docs-to-pdf/index.mjs docs/brd`	Renders `docs/brd/` to a PDF (this project's own documentation tooling — see `TRD-04`)

Not Found in Codebase: a `lint` script, a `format` script, or a pre-commit hook configuration (no `.husky/`, no `lint-staged` config found).

11.2 Test suite inventory (Fact — run fresh for this document, not quoted from memory)

`npm test` → **41 test files, 950 tests, all passing** (70.46s). `npx tsc --noEmit` → **0 errors.** Full file list (`src/test/*.test.{ts,tsx}`):

```
acceptance-2b.test.ts       ai.test.ts                  ai-draft-render.test.tsx
ai-model-access.test.ts     ai-providers.test.tsx       ai-run-task.test.ts
ai-tasks.test.ts            ai-task-settings.test.ts    ai-task-ui.test.tsx
ai-ui.test.tsx              app.test.tsx                backup.test.ts
blocks.test.ts              brand.test.ts               brief-checks.test.ts
brief-generate.test.ts      brief-misc.test.ts          brief-orchestrate.test.ts
brief-slots.test.ts         brief-storage.test.ts       brief-store.test.ts
brief-validate.test.ts      checks.test.ts              contrast.test.ts
dark.test.ts                dnd.test.ts                 email-defaults.test.ts
image-audit.test.ts         image-library.test.ts       merge.test.ts
plaintext.test.ts           presets.test.ts             profile-settings.test.tsx
render.test.ts               rte.test.tsx                saved-blocks.test.ts
storage.test.ts              store.test.ts                user.test.ts
utm.test.ts                  versions.test.ts
```

11.3 Test technique worth noting

`storage.test.ts`'s quota-path coverage stubs `Storage.prototype.setItem` to throw `QuotaExceededError` a controlled number of times, proving the prune-then-retry sequence in `writeSafe()` genuinely recovers rather than merely being read as correct from the source. `ai.test.ts`/`ai-*` files stub `fetch` entirely —

no test anywhere makes a real network call to OpenAI, confirmed by the suite passing with no API key configured in this environment.

11.4 Known test-suite performance note (Fact, from the suite's own output)

```
Environment  jsdom was created 41 times · 76% of tracked time
             create it once per worker with pool: 'vmThreads' (keeps per-file
             isolation) or isolate: false (shares it across files)
```

This is Vitest's own advisory output, not a failure — the suite passes in full; it is a possible future speed improvement, not a current defect.

Part 4 of 4 — Sections 12–16: Security, Performance, Risks, Developer Workflow, Glossary

> Continues from `TRD-03-ai-and-components.md`.

---

12. Security Architecture

12.1 Authentication/authorization: Not Found in Codebase

No login, session, password, OAuth, or JWT exists anywhere in `src/`. `src/lib/user.ts`'s own header comment: *"no login, no roles, an internal tool's substitute for an account."* `canEditBrand(user, brand)` / `getAccessibleBrands(user, brands)` are present as **named future seams** (`src/lib/brand.ts`) that currently return `true` / every active brand unconditionally.

12.2 Input sanitization

```
// src/lib/util.ts — safeUrl(): the one scheme gate every href/src passes through at render
time
// Allowed: https?:, mailto:, tel:, #, //, {{, *|, or 
// Everything else -> ''
```

Canvas content is sanitized via DOMPurify before being painted into the DOM (not an iframe); the Preview/export iframe is sandboxed **without** `allow-same-origin`/`allow-scripts`.

12.3 AI response validation as a security boundary

```
// src/lib/ai/validate.ts — magic-byte sniffing, never trusting declared content-type
// PNG: 89 50 4E 47 0D 0A 1A 0A   JPEG: FF D8 FF   WebP: RIFF....WEBP
// SVG refused outright, regardless of bytes — "a document that can carry script, not an
image"
// Size cap: MAX_IMAGE_BYTES = 10 * 1024 * 1024
```

The AI brief response passes through a dedicated sanitizer (`brief-validate.ts`) — HTML tags stripped, markdown tokens stripped, bare URLs stripped — **before** any of that text is ever handed to the HTML renderer.

12.4 Key exposure boundary

```
// src/lib/ai/provider.ts
function redactKeys(text: string, key: string): string {
  // strips the exact key (>= 8 chars), plus any sk-/rk-/org-/proj- prefixed
  // token, plus any "Bearer <token>" pattern, from provider-echoed error text
}
```

Applied inside every adapter's `classify()` before an error object is constructed — there is no code path where raw provider text reaches the UI unredacted. `maskKey()` renders a key for display as first-3 + bullets + last-4 characters (or all-bullets under 8 chars); nothing that writes to disk ever uses the unmasked value for display.

12.5 CORS as an architectural constraint, not an afterthought

The entire BYOK architecture rests on one empirically verified fact: `OPTIONS https://api.openai.com/v1/images/generations` (and the chat-completions equivalent) answers with `access-control-allow-headers: authorization,content-type` and `access-control-allow-origin: *`. Two direct engineering consequences: no `OpenAI-Organization` or telemetry header can ever be sent (it would fail the real preflight, not just this app's own validation), and the request/response shape is designed around exactly those two headers.

12.6 Privacy-relevant data flows

Data	Sent to	Trigger	Consent gate
Image generation prompt + style + optional brand color hex values	`api.openai.com`	"Send" in the AI image panel	None beyond having configured a key (no separate consent toggle for images)
Brief fields + brand name + brand voice guidance	`api.openai.com`	"Agree and generate" / "Generate"	Explicit, per-provider, one-time consent notice (`text-consent.ts`) — distinct from image-generation's lack of one
API key	`api.openai.com` only, as `Authorization: Bearer`	Every AI call	N/A — never sent anywhere else
Everything else (designs, brands, saved blocks, images)	Nowhere, unless the user explicitly exports/backs up a file	Manual export action only	N/A

---

13. Performance Characteristics

Mechanism	Value	Where
Checks re-run debounce	500ms, also forced on a whole-design load	`App.tsx`
Auto-save debounce	2000ms, plus a `beforeunload` flush	`App.tsx`
Undo coalescing window	900ms (`COALESCE_MS`)	`store.ts`
Undo/redo stack cap	100 steps, in-memory only (not persisted)	`store.ts` (`HISTORY_LIMIT`)
AI minimum request gap	2000ms between calls for the same task	`src/lib/ai/provider.ts`
AI request timeout	90,000ms, honoring both an internal timer and the caller's `AbortSignal`	`src/lib/ai/provider.ts`
Version-history cap	20 per email; quota-pressure pruning removes 10 globally-oldest at a time	`versions.ts`, `storage.ts`
Image-library re-probe	Every item, on every library open (no batching)	`ImageLibrary.tsx` — documented as a known scaling limit "at hundreds of images"

Not Found in Codebase: any load-testing artifact, Lighthouse/Web-Vitals budget, bundle-size budget, or code-splitting/lazy-loading configuration beyond Vite's own defaults.

14. Technical Risks, Debt & Constraints

Item	Category	Detail
Hardcoded AI model lists	Technical debt	Model ids (`gpt-image-1-mini`, `gpt-4.1-mini`, etc.) are literal string arrays in `src/lib/ai/openai.ts`/`openai-text.ts`/`tasks.ts`; a provider renaming or retiring a model (already happened once — `gpt-image-1` stopped existing) requires a manual code change, with no automated detection
No automated pruning for templates/brands/images/saved blocks	Technical debt	Only version history is auto-pruned under `localStorage` quota pressure; everything else requires the user to manually delete something once genuinely full
Two independent "1600px" constants	Technical debt (Inference — likely unintentional duplication)	`MAX_IMAGE_WIDTH` (`checks.ts`) and `VERY_LARGE_WIDTH` (`image-audit.ts`) are the same number, defined separately — a latent risk if one is ever tuned without the other
Single registered AI provider	Constraint	The fallback-chain feature's data model supports multiple providers, but only one (OpenAI) is implemented — the "fallback" today is always same-provider, different-model
No image-hosting adapter	Constraint (deliberate)	`ImageHosting.configured === false`; every AI-generated image exports as a placeholder until manually rehosted externally
No CI pipeline evidence	Not Found in Codebase	No CI config file was found in the repository; whether `npm test`/`npm run build` run automatically on any event is unconfirmed either way within this review's scope
No dependency-vulnerability scanning configuration found	Not Found in Codebase	No `.github/dependabot.yml`, no `npm audit` CI step, no equivalent found
Task-settings staleness across simultaneously open panels	Documented, accepted limitation	Changing a task's model in Settings while the AI image panel is also open does not live-update that panel's header until an unrelated re-render — deliberately not fixed with a shared subscription, per the code's own comment, as "disproportionate to one rare case"

15. Developer Environment & Workflow

15.1 Getting started (Fact — `README.md`)

```
npm install
npm run dev        # http://localhost:5173
npm test           # unit + component tests (vitest)
npm run build      # type-check + production build into dist/
```

15.2 This repository's own documentation-build tooling

A second, independent piece of tooling lives in `scripts/docs-to-pdf/` — a reusable script (not part of the application bundle, not shipped in `dist/`) that renders a folder of numbered Markdown doc parts (such as `docs/brd/`, `docs/prd/`, or this TRD) into one PDF, with Mermaid diagrams rendered to inline SVG via `@mermaid-js/mermaid-cli` and the page printed via `puppeteer` — both sharing one Chromium install (`puppeteer` is `@mermaid-js/mermaid-cli`'s own peer dependency, so only one browser is ever downloaded). Usage:

```
node scripts/docs-to-pdf/index.mjs <folder> [--no-appendix] [--out <path>] [--html <debug-path>]
npm run docs:pdf   # shorthand for the docs/brd folder specifically
```

It is read-only with respect to its Markdown inputs — it never writes back to any `.md` file — and performs a key-shaped-string scan over the merged text before producing any output, aborting rather than embedding a possible secret.

15.3 Environment variables

Not Found in Codebase: any `.env` file, `.env.example`, or `import.meta.env.VITE_*` usage in `src/` — confirmed by grep. The AI API key is entered through the UI at runtime and is never read from an environment variable or build-time constant.

---

16. Glossary (technical terms)

Term	Meaning
`commit()`	The one internal function through which every structural/value design mutation flows (`store.ts`), pushing an undo step and clearing redo
Coalescing	Merging rapid same-field edits within a 900ms window into a single undo step
`resolveDesign()` **/ resolve at the boundary**	Converting every `brand.*` token in a design to its literal value, exactly once, immediately before render/export
`normalizeDesign()`	The single gate through which any untrusted/stored JSON becomes a valid `Design`, used by every load path (draft, template, version, backup)
`writeSafe()`	The sole `localStorage` writer outside `storage.ts`'s own internals; implements prune-then-retry quota recovery
Slot	An AI-fillable text field in the brief-generation pipeline, derived by walking a preset's actual rendered block tree rather than hand-maintained per preset
`runTask()`	The single call-site function for any AI request; owns per-task routing, in-flight guarding, and fallback-on-error
`redactKeys()`	Strips key-shaped substrings from provider-echoed error text before it can reach the UI
Draft (AI image)	An IndexedDB-held `DraftImage` record; the `Design` object only ever stores its id (`aiDraftId`), never its bytes
Schema version	A per-category (not global) integer stamped on persisted records, letting `normalize*()` fill gaps in older records rather than rejecting them
CORS preflight	The `OPTIONS` request browsers send before a cross-origin call with custom headers; its allow-list (`authorization,content-type` only) is the architectural reason no backend proxy exists for AI calls

Appendix A: Verification Notes

Every **Inference** and **Not Found in Codebase** statement across `TRD-01-architecture.md`–`TRD-04-security-risks-and-appendix.md`, by section, with the reasoning behind each tag. Anything not listed here is a **Fact** — stated directly in source code, config files, tests, or the existing `docs/brd/` / `docs/prd/`. The test/type-check numbers quoted in this TRD (Part 3 §11.2) were run fresh, not copied from the BRD or PRD.

Part 1 (TRD-01-architecture.md)

§1 System Architecture Overview

- **Inference** — "Why this shape" (§1.2). *Reasoning:* the CORS-preflight fact itself is a direct quote/restatement of `docs/PROGRESS.md`'s own Feature 5A notes (a Fact); concluding that this is *why* the no-backend architecture was chosen is this document's own interpretive connection, clearly labeled as such.
- **Not Found in Codebase** — Any CI/CD config, Dockerfile, infrastructure-as-code, hosting config, or deployment script, and specifically which host (if any) serves the built `dist/` bundle in practice. *Reasoning:* none of these artifacts exist in the repository; a static build output existing is a Fact, but its actual deployment destination is unconfirmable from the repo alone.

§2 Technology Stack & Dependencies

- **Inference** — "Why each major dependency exists" (§2.2). *Reasoning:* `package.json` lists the dependency (Fact); the stated purpose for each is drawn from how it is actually used elsewhere in the codebase (confirmed during the BRD/PRD research passes), not from a dependency-rationale comment in the code itself (none exists).
- **Not Found in Codebase** — A split `tsconfig.node.json`/`tsconfig.app.json`; a `lint` script. *Reasoning:* one unified `tsconfig.json` was confirmed; `package.json`'s `scripts` object has no `lint` key.

§3 Project & Module Structure

- **Inference** — The module dependency-direction diagram (§3.4). *Reasoning:* no dependency graph is generated or documented anywhere in the repo; this is a reconstruction from the responsibility map and the known, explicit `backup.ts`-never-imports-`key-store.ts`/`drafts.ts` guarantee (a Fact, confirmed by reading that file's own imports).

§4 Data Model & Type System

- No new Inference/Not-Found tags — every interface quoted is a direct restatement of `src/types.ts`, cross-checked against the same data model already verified during the BRD's research pass (field

names/types unchanged, confirmed by the matching test-suite results at the top of this TRD's Part 3).

Part 2 (TRD-02-runtime-and-persistence.md)

- No new Inference/Not-Found tags. Every code excerpt (store shape, `resolveForOutput` order, `writeSafe` logic, the Checks rule table, the contrast math description) restates logic already directly read from source during this TRD's and the BRD's research, not inferred from indirect evidence.

Part 3 (TRD-03-ai-and-components.md)

§9 AI Integration Architecture

- No new Inference/Not-Found tags beyond those already carried over from the BRD's own AI research pass (e.g., the CORS-preflight fact, the error-classification table) — all restated as Facts with their original sourcing intact.

§10 Component Architecture

- No new Inference/Not-Found tags — the "no router" finding, the callback-registry pattern, and the shared-`SettingsView`-component fact are all direct code observations, cross-confirmed during the BRD's screen-inventory research.

§11 Build, Test & Tooling

- **Not Found in Codebase** — A `lint`/`format` script, or any pre-commit hook configuration (`.husky/`, `lint-staged`). *Reasoning:* none found by direct inspection of `package.json` and the repository root for this document.
- **Fact, called out for emphasis rather than tagged as Inference:** the 41-file/950-test/0-type-error numbers in §11.2 were obtained by running `npm test` and `npx tsc --noEmit` fresh, specifically for this TRD, not copied from the BRD/PRD's own (matching) numbers — stated explicitly so a reviewer doesn't need to assume staleness.

Part 4 (TRD-04-security-risks-and-appendix.md)

§12 Security Architecture

- **Not Found in Codebase** — Any authentication/authorization mechanism (restated from the BRD/PRD's own prior finding, re-verified for this document via the same `src/lib/user.ts` comment and grep sweep).

§13 Performance Characteristics

- **Not Found in Codebase** — Any load-testing artifact, Lighthouse/Web-Vitals budget, bundle-size budget, or non-default code-splitting configuration. *Reasoning:* none found in `vite.config.ts` or elsewhere in the repository.

§14 Technical Risks, Debt & Constraints

- **Inference** — Labeling the duplicate "1600px" constants (`MAX_IMAGE_WIDTH` in `checks.ts`, `VERY_LARGE_WIDTH` in `image-audit.ts`) as "likely unintentional duplication." *Reasoning:* both constants and their numeric equality are Facts (directly read from source); whether the duplication was ever a deliberate choice vs. an oversight is not stated anywhere in code comments, so the characterization is explicitly hedged as Inference.
- **Not Found in Codebase** — Any CI pipeline evidence, or any dependency-vulnerability-scanning configuration (Dependabot, `npm audit` in CI, Snyk, or equivalent). *Reasoning:* no such file or config found in the repository; explicitly stated as "unconfirmed either way," not as "does not exist in any environment," since a CI pipeline could in principle be configured entirely outside the repository (e.g., in a hosting platform's dashboard) with no trace in the codebase.

§15 Developer Environment & Workflow

- **Not Found in Codebase** — Any `.env` / `.env.example` file or `import.meta.env.VITE_*` usage. *Reasoning:* confirmed by grep across `src/`; the AI key is entered at runtime through the UI, never read from a build-time or environment variable.

§16 Glossary

- No new Inference/Not-Found tags — restates terms already defined and sourced earlier in this document.

Summary count

Tag	Approximate count across all 4 parts
Inference	~7 distinct statements/sections
Not Found in Codebase	~9 distinct statements/sections

Lower absolute counts than the BRD or PRD, consistent with a TRD's narrower scope: most of what a TRD covers (architecture, types, storage keys, rule tables) is directly readable from source with no inference required. The genuine gaps cluster exactly where you'd expect for a project with no deployment/ops artifacts committed to the repository: **deployment destination, CI/CD, dependency scanning, and environment-variable configuration** — all absent, and stated as such rather than assumed either way.